

MoneyTracker V1 - Feature Specification

3-Month Development Timeline (May - August 2026)

GSoC-Ready Android Portfolio Project

Project Overview

Goal: Build a functional expense tracking app that demonstrates Android development skills for GSoC 2027 applications.

Tech Stack: - Language: Kotlin - UI: Jetpack Compose (Material 3) - Database: Room (SQLite) - Architecture: MVVM - Charts: MPAndroidChart OR Compose Canvas - Version Control: Git + GitHub

Timeline: 3 months of focused development (1.5-2 hours daily)

Output: Clean, well-architected codebase with regular GitHub commits showing growth and skill development.

Core Architecture Requirements

MVVM Pattern (Non-Negotiable)

- **Model:** Room entities and DAOs
- **ViewModel:** Business logic, LiveData/StateFlow
- **View:** Jetpack Compose screens

Package Structure

```
app/src/main/kotlin/com/yourname/moneytracker/  
├── data/  
│   ├── local/  
│   │   ├── entities/  
│   │   ├── dao/  
│   │   └── database/  
│   └── repository/  
├── ui/  
│   ├── screens/  
│   ├── components/  
│   └── theme/  
├── viewmodel/  
└── utils/
```

Git Hygiene

- Meaningful commit messages: "Add transaction entry form with validation"
 - Logical commits (one feature per commit)
 - Regular pushes (daily if possible)
 - README.md with setup instructions and architecture explanation
-

MUST-HAVE Features (Priority 1)

These features demonstrate core Android skills needed for GSoC. Build them in order.

1. Room Database Setup ★

What: SQLite database using Room persistence library

Technical Details: - Entities: Transaction, Category - DAOs: TransactionDao, CategoryDao - Database class with singleton pattern - Type converters for Date and TransactionType enum

Database Schema:

Transaction Table: - id: Long (Primary Key, AutoGenerate) - amount: Double - type: TransactionType (EXPENSE/INCOME) - categoryId: Long (Foreign Key) - date: Long (Unix timestamp) - note: String (nullable, max 200 chars) - createdAt: Long

Category Table: - id: Long (Primary Key, AutoGenerate) - name: String - color: Int (color resource) - emoji: String (nullable) - orderIndex: Int - isArchived: Boolean

Why It Matters for GSoC: - Shows data persistence understanding - Demonstrates SQL/query skills - Proves you can design normalized schemas

Estimated Time: Week 2-3 (after Kotlin basics)

2. Expense & Income Entry Screen ★★★

What: Form to add new transactions

UI Components: - Amount input field (numeric keyboard) - Type selector (Expense/Income toggle or radio buttons) - Category dropdown/picker - Date picker (default: today, allow backdating) - Optional note field (200 char limit) - Save button

Business Logic: - Input validation (amount > 0, category selected) - Insert transaction into Room database - Navigate back to dashboard on success - Show error toast on validation failure

ViewModel:

```
class EntryViewModel(private val repository: TransactionRepository)
: ViewModel() {
    val categories: LiveData<List<Category>>

    fun saveTransaction(amount: Double, type: TransactionType,
        categoryId: Long, date: Date, note: String?)
```

```

        private fun validateInput(...): Boolean
    }

```

Why It Matters for GSoC: - Core user interaction pattern - Form validation logic - State management in Compose - Repository pattern usage

Estimated Time: Week 3-4

3. Custom Categories Management ★★

What: CRUD operations for expense/income categories

Features: - View list of existing categories - Add new category (name, color picker, optional emoji) - Edit category name/color/emoji - Delete category (only if no transactions linked) - Archive category instead of delete if transactions exist - Reorder categories (drag-and-drop or up/down buttons)

Default Categories (Pre-loaded on first launch): - Food 🍔 (Orange) - Transport 🚗 (Blue) - Bills 💰 (Red) - Shopping 🛒 (Purple) - Health 🏥 (Green) - Entertainment 🎮 (Pink) - Salary 💵 (Teal) - Other ★ (Gray)

UI: - Category list screen (LazyColumn) - Add/Edit dialog or bottom sheet - Color picker (20-color grid) - Emoji picker (simple text input or basic emoji selector)

Why It Matters for GSoC: - Demonstrates CRUD mastery - Complex UI interactions - Database relationships (foreign keys) - Edge case handling (can't delete used categories)

Estimated Time: Week 4-5

4. Dashboard Screen ★★★

What: Home screen showing monthly financial summary

Display Elements: - Current month and year (e.g., "May 2026") - Month navigation (< and > buttons) - Total Income (green card) - Total Expenses (red card) - Net Balance (large, centered, color-coded: green if positive, red if negative) - Recent transactions list (last 10-15)

Data Aggregation:

```

// Repository function
fun getMonthSummary(year: Int, month: Int): Flow<MonthSummary>

data class MonthSummary(
    val totalIncome: Double,
    val totalExpenses: Double,
    val netBalance: Double,
    val recentTransactions: List<Transaction>
)

```

UI Features: - Pull-to-refresh to reload data - Tap transaction to view/edit details - Empty state when no transactions ("Start tracking your expenses!")

Why It Matters for GSoC: - Shows data aggregation skills - Flow/LiveData usage - Complex UI layout - Real-time updates when data changes

Estimated Time: Week 5-6

5. Transaction List Screen ★★

What: Scrollable list of all transactions with filtering

Display: - LazyColumn of transaction items - Grouped by date (section headers: "Today", "Yesterday", "May 10, 2026") - Each item shows: category icon, amount, category name, optional note - Month filter dropdown (current month, last month, all time) - Type filter chips (All, Expenses, Income)

Interaction: - Tap to open edit screen - Long-press for quick delete (with confirmation) - Empty state when no transactions in filter

Room Query:

```
// DAO function
@Query("""
    SELECT * FROM transactions
    WHERE (:startDate IS NULL OR date >= :startDate)
    AND (:endDate IS NULL OR date <= :endDate)
    AND (:type IS NULL OR type = :type)
    ORDER BY date DESC
""")
fun getFilteredTransactions(
    startDate: Long?,
    endDate: Long?,
    type: TransactionType?
): Flow<List<Transaction>>
```

Why It Matters for GSoC: - Advanced Room queries with filters - Efficient list rendering (LazyColumn) - State management for filters - Grouped list implementation

Estimated Time: Week 6-7

6. Edit & Delete Transactions ★★

What: Modify or remove existing transactions

Edit Screen: - Pre-filled form with existing transaction data - All fields editable - Update button saves changes - Cancel button discards changes

Delete Functionality: - Delete button with confirmation dialog - Soft delete option (mark as deleted, don't actually remove) - OR hard delete (permanently remove from database)

Business Logic:

```
// ViewModel
fun updateTransaction(id: Long, amount: Double, type:
TransactionType,
                    categoryId: Long, date: Date, note: String?)
```

```
fun deleteTransaction(id: Long)
```

Why It Matters for GSoC: - Full CRUD cycle completion - Update/Delete SQL operations - Confirmation dialogs (UX best practice) - Data integrity handling

Estimated Time: Week 7

7. Basic Chart Visualization **

What: One chart type showing spending breakdown

Option A: Pie Chart (Recommended for V1) - Shows percentage of total spending per category - Color-coded by category colors - Tap slice to see exact amount and percentage - Monthly view (matches dashboard month)

Option B: Bar Chart - Daily spending totals across the month - X-axis: days 1-31 - Y-axis: total amount - Color distinction for expense/income

Implementation:

Using MPAndroidChart:

```
// In Compose, use AndroidView to embed MPAndroidChart
AndroidView(
    factory = { context ->
        PieChart(context).apply {
            // Configure chart
        }
    },
    update = { chart ->
        val entries = categoryData.map {
            PieEntry(it.amount.toFloat(), it.categoryName)
        }
        chart.data = PieData(PieDataSet(entries, ""))
        chart.invalidate()
    }
)
```

OR Using Compose Canvas (More Modern):

```
Canvas(modifier = Modifier.fillMaxSize()) {
    // Draw pie chart manually using drawArc
}
```

Why It Matters for GSoC: - Data visualization skills - Third-party library integration OR custom drawing - Complex UI rendering - Demonstrates analytical thinking

Estimated Time: Week 8-9

8. Monthly Navigation ★

What: Switch between different months to view historical data

UI: - Previous/Next month buttons on dashboard - Month/Year display (e.g., "May 2026") - Updates dashboard totals and recent transactions - Updates chart data - Can navigate to any past month - Can't navigate to future months

State Management:

```
// ViewModel
private val _selectedMonth = MutableStateFlow(YearMonth.now())
val selectedMonth: StateFlow<YearMonth> = _selectedMonth

fun navigateToPreviousMonth()
fun navigateToNextMonth()
```

Why It Matters for GSoC: - Date/time handling - State preservation across navigation - Query parameterization - Temporal data filtering

Estimated Time: Week 9

NICE-TO-HAVE Features (Priority 2)

Build these ONLY if you finish all Must-Have features with time remaining. They're polish, not core functionality.

9. PIN Lock Screen

What: Simple 4-6 digit PIN to lock app access

Features: - Set PIN on first launch (settings screen) - Lock screen shows on app open - Correct PIN unlocks to dashboard - Wrong PIN shows error, allows retry - "Forgot PIN" option resets app (clears all data - extreme but acceptable for V1)

Implementation: - Store PIN hash in SharedPreferences (use simple hashing) - Check on MainActivity onCreate - Lock after app backgrounds for >1 minute

Skip for V1: Biometric unlock (complex Android Biometric API)

Why It Matters for GSoC: - Security awareness - Activity lifecycle understanding - SharedPreferences usage

Estimated Time: Week 10 (if time allows)

10. Dark & Light Theme

What: Support system dark mode

Implementation:

```
// In theme/Theme.kt
@Composable
fun MoneyTrackerTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val colorScheme = if (darkTheme) DarkColorScheme else
```

LightColorScheme

```
MaterialTheme(  
    colorScheme = colorScheme,  
    typography = Typography,  
    content = content  
)  
}
```

What to Theme: - Background colors - Card colors - Text colors - Category colors (ensure visibility in both modes) - Chart colors

Why It Matters for GSoC: - Material Design 3 understanding - Dynamic theming - Android system integration

Estimated Time: Week 11 (if time allows)

11. Basic Settings Screen

What: Simple preferences screen

Settings to Include: - Currency symbol (₹, \$, €, etc.) - Default transaction type (Expense/Income) - PIN change option - Theme toggle (Light/Dark/System) - About section (version number, privacy statement)

Implementation: - Use Jetpack Compose settings components - Store preferences in DataStore (modern alternative to SharedPreferences)

Why It Matters for GSoC: - User preferences handling - DataStore usage - Settings UI patterns

Estimated Time: Week 11-12 (if time allows)

EXPLICITLY SKIPPING for V1

These are in your full MoneyTracker doc but NOT needed for GSoC demonstration. Save for later versions.

✗ SQLCipher Encryption

Why Skip: - Adds 2-3 weeks of complexity - Not needed to demonstrate Android skills - Can add in V1.5 after GSoC applications

✗ Multiple Chart Types

Why Skip: - One chart proves the concept - More charts = same skills repeated

✗ Animations & Polish

Why Skip: - Doesn't demonstrate new technical skills - Time better spent on architecture quality

✗ Home Screen Widget

Why Skip: - Complex Android component - V2 feature, not core functionality

✗ Budgets, Goals, Reminders

Why Skip: - All V2 features - V1 is about core tracker functionality

Technical Quality Checklist

Before calling V1 “done,” ensure:

Code Quality

- ✓ No magic numbers (use constants)
- ✓ Meaningful variable/function names
- ✓ Comments explaining complex logic
- ✓ No unused imports or code
- ✓ Consistent formatting (use ktlint or Android Studio formatter)

Architecture

- ✓ ViewModels never reference View/Context
- ✓ Repository pattern properly implemented
- ✓ UI state managed with StateFlow/LiveData
- ✓ No business logic in Composables

Error Handling

- ✓ Try-catch around database operations
- ✓ Input validation before saving
- ✓ Null safety (no !! operators unless absolutely necessary)
- ✓ Graceful empty states

Git Hygiene

- ✓ .gitignore includes build/, local.properties, *.iml
- ✓ No API keys or secrets committed
- ✓ Clear commit messages
- ✓ README.md with setup instructions

Testing (Bonus, Not Required)

- Unit tests for ViewModels (bonus points for GSoC)
 - Repository tests with fake data
 - UI tests for critical flows (nice to have)
-

Week-by-Week Breakdown

Weeks 1-2: Foundation

- **Learn:** Kotlin basics, Compose fundamentals
- **Build:** Project setup, Room database schema

- **Output:** Empty app with database working

Weeks 3-4: Core Entry

- **Build:** Entry screen, Category management
- **Output:** Can add transactions and categories

Weeks 5-6: Viewing Data

- **Build:** Dashboard, Transaction list
- **Output:** Can view and browse all data

Weeks 7-8: Editing & Visualization

- **Build:** Edit/delete, Basic chart
- **Output:** Full CRUD + visual insights

Weeks 9-10: Navigation & Polish

- **Build:** Monthly navigation, PIN lock (if time)
- **Output:** Complete V1 MVP

Weeks 11-12: Quality & Documentation

- **Build:** Bug fixes, code cleanup, README
 - **Output:** GSoC-ready GitHub repository
-

Success Criteria

By end of August 2026, you should have:

Technical Deliverables

1. ✓ Working Android app (builds and runs without errors)
2. ✓ All 8 Must-Have features implemented
3. ✓ Clean MVVM architecture
4. ✓ Room database with proper relationships
5. ✓ At least one data visualization (chart)

GitHub Portfolio

1. ✓ 80-100 commits over 3 months (shows consistency)
2. ✓ Clear commit history showing feature progression
3. ✓ README.md explaining project, setup, and architecture
4. ✓ Screenshots in README showing app features

Skills Developed

1. ✓ Kotlin fluency (ready to read other Kotlin codebases)
2. ✓ Jetpack Compose proficiency
3. ✓ Room Database operations
4. ✓ MVVM pattern understanding
5. ✓ Git workflow basics

GSoC Readiness

1. ✓ Can confidently discuss your architecture decisions
 2. ✓ Can navigate a Kotlin Android codebase
 3. ✓ Ready to attempt “good first issue” in open source
 4. ✓ Portfolio demonstrates 3 months of consistent learning
-

Next Steps

1. **Share your college timetable** → We'll map out daily coding blocks
 2. **Set up development environment** → Android Studio, Git, GitHub account
 3. **Start learning Kotlin** → Kotlin Koans, official docs
 4. **Create GitHub repository** → Initialize project structure
 5. **Begin Week 1** → Database schema design and Room setup
-

Remember

This is a learning project, not a competition.

- Don't compare your V1 to polished Play Store apps
- Focus on clean architecture over fancy features
- Consistent progress > perfection
- Document your learning journey in commits and README

GSoC mentors want to see: - Growth over 3 months ✓ - Ability to learn independently ✓ - Clean, readable code ✓ - Genuine interest in Android development ✓

Your V1 + consistent open source contributions from September = strong GSoC 2027 candidate.

Document Version: 1.0

Last Updated: May 2026

Target Completion: August 2026

Next Phase: Open Source Contributions (September 2026)